90-MINUTE FINANCIAL RAG COPILOT LAB

A compact AI/ML engineering project aligned to an entry/intermediate financial-services AI engineering role

RAG -> Prompt Engineering -> FastAPI -> pytest -> Docker -> CI -> optional Vertex AI + Cloud Run

Tonight's objective: Finish one credible mini-project, not another giant platform build. Hard stop at 90 minutes; cloud deployment is optional if you are tired.

What this project gives you a flavour of

- Python application engineering: modules, typed API contracts, tests, Git and review discipline.
- RAG fundamentals: retrieval, context construction, grounded prompting, citations and refusal when evidence is insufficient.
- LLM integration through Vertex AI Gemini 2.5 Flash-Lite, chosen as a low-latency/cost-conscious model.
- Financial-services context: change risk, payment releases, incidents and governance - using synthetic policies only.
- Containerisation and CI: Docker plus GitHub Actions.
- Cloud-native deployment: optional Artifact Registry + Cloud Run configured to scale to zero.
- Communication: a reviewer-friendly README, architecture explanation, governance boundary and 60-second stakeholder summary.
- AI coding-tool hygiene: use an assistant for review/explanation, record what you accepted/rejected, and verify with tests.

Architecture

```
User question
  |
  v
TF-IDF retriever (local, deterministic)
  |  top-2 synthetic policy documents
  v
Grounded prompt + evidence boundary
  |
  v
Gemini 2.5 Flash-Lite on Vertex AI
  |
  v
FastAPI response: answer + sources

Delivery path: pytest -> Docker -> GitHub Actions -> optional Cloud Run
```

Why TF-IDF instead of a vector database: The point tonight is the RAG control flow, engineering discipline and LLM grounding. A vector DB/embedding service is an obvious next iteration, but it adds setup without improving the core lesson.

90-minute run sheet

Time	Stage	Evidence	Stop condition

0-10	Scaffold + synthetic policy corpus	Repo, venv, Git branch	Dependencies install
10-25	Retriever	Ranked source documents	Retrieval test green
25-45	RAG + prompt + API	/ask returns answer + sources	Local curl succeeds
45-58	Testing + AI-review workflow	pytest + AI_USAGE.md	Tests green
58-72	Docker + CI	Container + workflow	Container curl succeeds
72-82	Communication + review	README + PR checklist	Repo reviewer-ready
82-90	Optional GCP deployment	Cloud Run URL	/health and /ask work; then clean up

Stage 0 - prerequisites (before timer)

```
python3 --version
git --version
docker --version
gcloud --version   # only needed for optional cloud stage
```

- Use your normal AI coding assistant if you want, but never paste credentials, customer data, proprietary policy text or employer-internal material into it.
- For Vertex AI locally, run gcloud auth application-default login once if ADC is not already configured.

Stage 1 - scaffold and create the synthetic policy corpus (0-10 min)

```
mkdir financial-change-rag-copilot && cd financial-change-rag-copilot
git init
git checkout -b feature/rag-copilot
mkdir -p src tests data/policies prompts .github/workflows gcp
python3 -m venv .venv
source .venv/bin/activate
```

Create requirements.txt:

```
fastapi==0.116.1
uvicorn[standard]==0.35.0
scikit-learn==1.7.1
google-genai>=1.30.0
pydantic==2.11.7
pytest==8.4.1
httpx==0.28.1
```

```
pip install -r requirements.txt
```

Create four short Markdown documents in data/policies/. Use only synthetic content. Suggested topics:

- production-change-standard.md - owner, validation, rollback, change windows
- payments-release-policy.md - payment releases, backward-compatible schema changes, reconciliation
- incident-severity-guide.md - SEV1/2/3 impact and communication
- model-risk-note.md - evidence-only assistant, no fabricated approvals or regulatory claims

Checkpoint: git status shows your new files; no secrets or real internal documents are present.

Stage 2 - build retrieval (10-25 min)

Create src/retriever.py. This turns each policy document into TF-IDF features and ranks documents by cosine similarity.

```
from pathlib import Path
from dataclasses import dataclass
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

@dataclass
class Hit:
    source: str
    text: str
    score: float

class PolicyRetriever:
    def __init__(self, policy_dir: str = "data/policies"):
        paths = sorted(Path(policy_dir).glob("*.md"))
        self.sources = [p.name for p in paths]
        self.docs = [p.read_text(encoding="utf-8") for p in paths]
        self.vectorizer = TfidfVectorizer(stop_words="english", ngram_range=(1, 2))
        self.matrix = self.vectorizer.fit_transform(self.docs)

    def search(self, query: str, top_k: int = 2) -> list[Hit]:
        q = self.vectorizer.transform([query])
        scores = cosine_similarity(q, self.matrix)[0]
        idxs = scores.argsort()[::-1][:top_k]
        return [Hit(self.sources[i], self.docs[i], float(scores[i])) for i in idxs]
```

Create tests/test_retriever.py:

```
from src.retriever import PolicyRetriever

def test_payment_change_retrieval():
    r = PolicyRetriever()
    hits = r.search("payment database schema change rollback peak hours", top_k=2)
    names = {h.source for h in hits}
    assert "payments-release-policy.md" in names
```

```
python -m pytest tests/test_retriever.py -q
```

What you should understand: RAG has two distinct phases: retrieval decides what evidence enters the prompt; generation decides how the model uses that evidence. Bad retrieval cannot be rescued reliably by a better prompt.

Stage 3 - prompt engineering + Vertex AI LLM wrapper (25-35 min)

Create prompts/system.txt with a strict evidence boundary:

```
You are a financial-technology change-risk assistant. Answer ONLY from the supplied context.
If the context is insufficient, say: "Insufficient evidence in the supplied policy set."
Do not invent approvals, exceptions, controls, customer information, or regulatory conclusions.
Give a concise answer, then a short "Evidence" section naming the source documents.
```

Create src/llm.py:

```
import os
from google import genai
from google.genai import types

MODEL = os.getenv("GENAI_MODEL", "gemini-2.5-flash-lite")

def generate_answer(question: str, context: str) -> str:
    if os.getenv("MOCK_LLM", "false").lower() == "true":
        return "Mock answer generated from retrieved context."

    project = os.environ["GOOGLE_CLOUD_PROJECT"]
    location = os.getenv("GOOGLE_CLOUD_LOCATION", "global")
    client = genai.Client(vertexai=True, project=project, location=location)
    system = open("prompts/system.txt", encoding="utf-8").read()
    prompt = f"{system}\n\nCONTEXT:\n{context}\n\nQUESTION:\n{question}"
    response = client.models.generate_content(
        model=MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(temperature=0, max_output_tokens=350),
    )
    return response.text
```

Set the local Vertex AI environment:

```
export GOOGLE_CLOUD_PROJECT="YOUR_PROJECT_ID"
export GOOGLE_CLOUD_LOCATION="global"
export GENAI_MODEL="gemini-2.5-flash-lite"
gcloud auth application-default login
```

Prompt-engineering lesson: The useful part is not a fancy persona. It is the contract: use supplied context only, state insufficiency, forbid invented approvals, and require evidence naming. Temperature 0 reduces unnecessary variation for this policy-style use case.

Stage 4 - expose the RAG flow as an API (35-45 min)

Create src/api.py:

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
from src.retriever import PolicyRetriever
from src.llm import generate_answer

app = FastAPI(title="Financial Change RAG Copilot", version="1.0.0")
retriever = PolicyRetriever()

class AskRequest(BaseModel):
    question: str = Field(min_length=5, max_length=500)

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/ask")
def ask(req: AskRequest):
    hits = retriever.search(req.question, top_k=2)
    context = "\n\n".join(f"SOURCE: {h.source}\n{h.text}" for h in hits)
```

```
    answer = generate_answer(req.question, context)
    return {
        "answer": answer,
        "sources": [{"name": h.source, "score": round(h.score, 3)} for h in hits],
    }
uvicorn src.api:app --reload --port 8001
```

In a second terminal:

```
curl -X POST http://127.0.0.1:8001/ask \
  -H "Content-Type: application/json" \
  -d '{"question":"Can I deploy a payment schema change during peak hours without a rollback plan?"}'
```

Your answer should be conservative and should cite retrieved policy filenames. Try a question the corpus cannot answer, for example:

```
curl -X POST http://127.0.0.1:8001/ask -H "Content-Type: application/json" -d '{"question":"What is the FCA fine for this specific breach?"}'
```

Expected behaviour: The second question should not invent a regulatory answer. A safe system explicitly knows when the supplied evidence is insufficient.

Stage 5 - tests + AI coding-tool workflow (45-58 min)

Create tests/test_api.py. The test deliberately mocks the LLM so CI does not spend money or require cloud credentials:

```
import os
os.environ["MOCK_LLM"] = "true"
from fastapi.testclient import TestClient
from src.api import app

client = TestClient(app)

def test_health():
    assert client.get("/health").json() == {"status": "ok"}

def test_ask_returns_sources():
    r = client.post("/ask", json={"question": "Can I deploy a payment schema change in peak hours without rollback?"})
    assert r.status_code == 200
    body = r.json()
    assert body["answer"]
    assert len(body["sources"]) == 2
python -m pytest -q
```

Now use your AI coding assistant once as part of the workflow. Give it a narrow review task, such as:

Suggested AI-review prompt: Review src/api.py and src/retriever.py for testability, failure modes and unsafe assumptions. Do not rewrite the project. Give at most five findings and explain how I can verify each one.

Create AI_USAGE.md and record: the prompt, one suggestion you accepted, one you rejected, and how you verified the accepted change. This turns "I use AI coding tools" into an auditable engineering workflow rather than blind code generation.

Stage 6 - containerise and add CI (58-72 min)

Create Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src ./src
COPY data ./data
COPY prompts ./prompts
ENV PORT=8080
EXPOSE 8080
CMD ["sh", "-c", "uvicorn src.api:app --host 0.0.0.0 --port ${PORT}"]
```

```
docker build -t financial-change-rag:local .
docker run --rm -p 8001:8080 \
 -e GOOGLE_CLOUD_PROJECT="$GOOGLE_CLOUD_PROJECT" \
 -e GOOGLE_CLOUD_LOCATION=global \
 -v "$HOME/.config/gcloud:/root/.config/gcloud:ro" \
 financial-change-rag:local
```

The gcloud credential mount above is for a local lab only. Do not bake credentials into the image. In another terminal, call /health and /ask again.

Add .github/workflows/ci.yml:

```
name: CI
on:
 push:
 pull_request:
jobs:
 test-and-build:
  runs-on: ubuntu-latest
  steps:
   - uses: actions/checkout@v4
   - uses: actions/setup-python@v5
     with:
       python-version: "3.12"
   - run: pip install -r requirements.txt
   - run: pytest -q
   - run: docker build -t financial-change-rag:ci .
```

Engineering signal: CI uses the mock LLM path: tests are deterministic and free, while the real LLM integration is exercised manually/in an integration environment.

Stage 7 - code review + stakeholder communication (72-82 min)

Add REVIEW_CHECKLIST.md with checks for tests, grounding, source citations, secrets, Docker build, limitations and cleanup. Then write the first screenful of README.md to answer:

- What business problem does this solve?
- What is the RAG architecture?
- What makes an answer grounded?
- What happens when evidence is insufficient?
- How do I run tests and Docker?
- What data/governance boundaries apply?
- What would I change before production?

Practice this 60-second stakeholder explanation:

Stakeholder narrative: This is a decision-support assistant for change-risk questions. It retrieves the most relevant approved-style policy text before calling the LLM, constrains the model to that context, and returns the source names with the answer. The demo uses synthetic data and is not an approval engine. Tests mock the paid model call, while the real integration can run through Vertex AI.

Commit the milestone:

```
git add .
git commit -m "build grounded financial change RAG copilot"
```

Stage 8 - optional 8-minute cloud-native deployment (82-90 min)

Only do this if you still have energy. Cloud Run is a useful small-project target because it can scale to zero when idle. Use Artifact Registry and a dedicated runtime service account with Vertex AI access.

```
export PROJECT_ID="YOUR_PROJECT_ID"
export REGION="europe-west1"
bash gcp/deploy.sh
```

The reference solution includes gcp/deploy.sh. After deployment:

```
SERVICE_URL=$(gcloud run services describe financial-change-rag --region="$REGION" --format="value(status.url)")
curl "$SERVICE_URL/health"
curl -X POST "$SERVICE_URL/ask" -H "Content-Type: application/json" \
  -d '{"question":"What controls should exist before a payment schema change?"}'
```

Then clean up immediately:

```
gcloud run services delete financial-change-rag --region="$REGION" --quiet
gcloud artifacts repositories delete rag-lab --location="$REGION" --quiet
```

Cost guardrail: Use only a handful of Gemini calls, max one Cloud Run instance, min zero, then delete the service and registry. Do not leave a cluster or always-on VM running.

Definition of done

- ☐ Retriever test passes and ranks the payment policy for a payment-change query.
- ☐ /ask returns an answer plus source names.
- ☐ An out-of-scope question produces an insufficient-evidence response rather than an invented fact.
- ☐ pytest passes without cloud credentials or paid LLM calls.
- ☐ Docker image builds and serves /health.
- ☐ GitHub Actions runs tests and builds the image.
- ☐ AI_USAGE.md records one reviewed AI-assisted development interaction.
- ☐ README explains architecture, limitations, governance and production next steps.
- ☐ Optional: Cloud Run endpoint works and is cleaned up afterward.

How this maps to the role

Role requirement	Lab evidence	What to deepen next
Python + software engineering	Modules, FastAPI, pytest, Git, PR checklist	Typing, linting, integration tests, package layout
LLM / RAG / prompt engineering	Retrieval + grounded Gemini prompt + refusal boundary	Embeddings/vector DB, reranking, evaluation set
AI coding tools	AI_USAGE.md with accept/reject/verify discipline	IDE assistant, secure enterprise tooling, evaluation

CI/CD + containers + cloud	GitHub Actions, Docker, optional Cloud Run	CD workflow, IaC, Kubernetes
Financial services exposure	Synthetic change/payment/incident policy corpus	Controls, model risk, auditability, data governance
Communication	README + stakeholder narrative + limitations	Architecture decision records, demos, design reviews
Databricks/Kubernetes familiarity	Not the focus of this 90-min lab	Add Databricks Vector Search or GKE in iteration 2

Good next iterations - do not do tonight

- Replace TF-IDF with embeddings + FAISS/pgvector/Databricks Vector Search and measure retrieval quality.
- Create a 20-question evaluation dataset and score retrieval hit-rate, groundedness and refusal quality.
- Add prompt-injection tests and document-level access-control filtering.
- Add structured logging, latency/token metrics and cost monitoring.
- Deploy to GKE or Databricks Apps/Model Serving and manage secrets through workload identity.
- Add a simple agent tool, e.g. fetch a synthetic change record by ID, with explicit authorization boundaries.

Portfolio/CV wording after you finish

CV bullet: Built a financial-services RAG copilot using Python, FastAPI and Vertex AI Gemini; implemented deterministic policy retrieval, grounded prompting with source attribution and insufficient-evidence handling, automated pytest/Docker CI, and an optional scale-to-zero Cloud Run deployment using synthetic governance data.

Do not oversell it as production experience. In interviews, describe it as a portfolio project that gave you hands-on exposure to the architecture, failure modes and governance questions you would expect in an enterprise AI team.